

Kurv

Kurv SDLC w Containers

Day 3 - DevOps

12-Feb-2026

Pythian

Agenda

- Overview
- What is a container
- Best Practices
- Development of Application
- Docker and Docker Compose
- Questions & Next Steps



Goals of the next 3 days

- A fully functional CI/CD pipeline.
 - Application being built from a repo
- A sample application deployed to a cloud environment.
 - Application once built will be deployed
- A roadmap for implementing a modern SDLC in your organization.
 - What could that roadmap look like to you
- All workshop materials, including presentations and code samples.
 - Repo shared

DevOps Cloud Ops Platform Engineering?

Platform Engineering

Configures and manages the platform's golden paths, including baseline infrastructure and application configurations.

Devops/SRE/Cloud Ops

Provides input on infrastructure configuration standards and ensures that cloud infrastructure is set up securely, efficiently, and in line with the platform team's needs. Responsible for cloud infrastructure management, including compute, storage, and networking, ensuring high availability, redundancy, and disaster recovery.

<https://platformengineering.org/blog/platform-engineering-vs-devops-vs-sre>

What is our infrastructure going to look like?

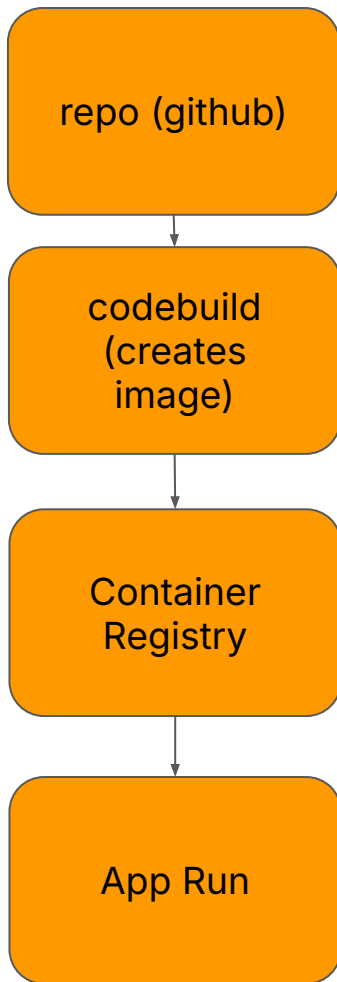
A Basic Container Host

We are keeping this simple, effectively we will have:

Container registry (Artifact Registry) (ECR), which will be a hand off from the build process to infrastructure.

App Runner, which is a fully managed container runner in AWS.

Codebuild, which will detect changes and build our container on demand



Deploy our application

Container Image Registry

Elastic Container Registry

- This is where container images are stored
 - Before Deployment
 - For Rollback

For this we only need to create the repository end point for the given image.

<https://aws.amazon.com/ecr/>

Command:

```
echo "Creating ECR repository for App Runner..."  
  
aws ecr create-repository --repository-name  
<ECR_REPO_NAME>
```

Deploy our application

Build Pipeline Runner

CodeBuild

Code build is a containerized Continuous Integration service that will compile source code, runs tests and produce a software package.

After configuration it can be used to run commands or scripts to via the buildspec.yml file.

<https://aws.amazon.com/codebuild/>

Commands:

Create a role that has the rights necessary

- `aws iam create-role`
- `aws iam put-role-policy`

Connect AWS to GitHub

- `aws codebuild import-source-credentials`

Create the Project that will run the code:

- `aws codebuild create-project`

Create the event hook to know when a change has occurred

- `aws codebuild create-webhook`

Deploy our application

Build Pipeline Runner

buildspec.yml

This is the list of commands/scripts you want to run as part of the pipeline. As long as the system has the rights it can do most anything you want from a CLI type of grouping.

This is running in a container which is temporary in nature (stateless) so anything that is required needs to be accessible.

- Environment variables
- In the GitHub repository (data store)
- In another location (S3 bucket) that can be accessed by the codebuild pipeline.

<https://malsouli.medium.com/10-smart-ways-to-use-aws-codebuild-22a8ee0d9302>

Commands:

Most anything that can run in a container can run here (installations pending).

In our case there are 3 main commands:

- docker build - Building the docker image (similar to docker compose from yesterday)
- docker tag - applying the tags we want to see/use in ECR
- docker push - pushing the image to ECR

Deploy our application

Infrastructure Host

App Runner

App Runner is a container host that comes out of the box with Load Balancing, HTTPS encryption and can generally automatically connect with other AWS services

<https://aws.amazon.com/apprunner/>

Container

Create a role that has the rights necessary

- aws iam create-role
- aws iam put-role-policy

Create the container service

- aws apprunner create-service

Re-Deploy our application

Making Changes

Feels like a lot of work.

1. Application is now documented and can be deployed easily to another environment.
2. Changes can be made to the application source code in GitHub and that will automatically deploy the application
3. Minor changes can be made directly by the development as long as the only change is in the buildspec.yml file

Container

- Make a change in the github repo and follow the deployment process.

CloudFormations

Re-Deploy our application - w Cloud Foundations

Making Changes

Cloud Foundations

- yaml file instead of bash scripts
- basically the same information packaged differently

```
echo "Creating ECR repository for App Runner..."
aws ecr create-repository --repository-name
kurv-my-app-frontend-repo-ds
```

Becomes:

```
ContainerRepositoryFrontend
Type: AWS::ECR::Repository
Properties:
  RepositoryName: !Sub "${ProjectName}-frontend-repo"
```

- Can (will) roll back if errors occurs
 - This can be more problematic than you think

Container

- Better for validations
- Better for naming conventions
- Better understanding what is there at a glance
- Can fully create and delete an environment without issue
- Files can become large
- Often needs to be split up due to stages of completion

CodeBuild/CodeDeploy/CodePipeline

What's the Difference between them

CodeBuild

- Builds the code and puts it somewhere ready to be deployed. Often this will be a container registry or and S3 bucket.

CodeDeploy

- Instructions for "how to deploy". Links code artifacts to an environment and instructs how to deploy it.

CodePipeline

- Orchestrates the whole thing. Basically can combine multiple CodeBuild and CodeDeploy items together.
 - Enforce approvals if needed
 - Control Version if needed (i.e. not latest)

CloudFormation can be both a creator and consumer of CodePipeline.

- As automation increases you can create a step in the pipeline to deploy the CloudFormation yaml
- Creating an environment on the fly
- Updating environments as you update through environments (QA, UAT, Production, DR)
- Updating scaled environments across regions
 - Large deployments which may be in multiple AWS data centers.

Roadmap

What Stuff

What are all of the "things" that you need to run the app?

List of "stuff" here :-)

Who Owns What?

Who are the owners of what, and what can others do with it.

What are the “teams” that are part of this currently?

- Network
- DBA
- Development
- Server Management
- QA?

Effectively for each “thing” on the previous page what team “owns it” and what other teams need to interact with it.

i.e.

Server Management Owner App Runner

Development needs to be able to see the configuration and logs from App Runner for debugging

QA needs the URL for app runner to test the system.

What are the Team Needs?

Who are the owners of what, and what can others do with it.

What are the “teams” that are part of this currently?

- Network
- DBA
- Development
- Server Management
- QA?

Effectively for each “thing” on the previous page what team “owns it” and what other teams need to interact with it.

i.e.

Server Management Owner App Runner

Development needs to be able to see the configuration and logs from App Runner for debugging

QA needs the URL for app runner to test the system.

repeat as necessary

Can we identify Gaps

Where are there Gaps today

This is early, but given what we know where are there gaps in the current process or a projected process either via the tools, or security or both?

- PRs are your main gate for everything that is in source control, which ultimately means everything that goes to production.
- The more you automate the more this will be true. This supports
 - Shift Left thinking (earlier is better)
 - for those things that must be true
 - Shift Right thinking (fail fast)
 - for those things that require feedback

Questions & Next Steps

A few basics:

- Finalize Pricing
- Align on potential start date timing



Thank you